

rain-sdk-v2 — Road to 9/10 (Non-Breaking Remediation Plan)

Baseline: 2.3.0 (scored 8.5/10, verified 21 Sep 2026)

Target release: `2.3.1` (patch — additive/corrective only)

Hard constraint: every change below is designed to be **non-breaking for existing integrations**. No signature removals, no behavioral change for any code path that works correctly today.

[CRITICAL] Blockers — required for 9

1. F-01 — Guard the reverting order reads **(non-breaking by construction)**

`getUserActiveBuyOrders` / `getUserActiveSellOrders` currently revert on **every** deployed market. No working integration exists, therefore guarding them **cannot break anyone's flow** — it only changes **where** the failure happens (dev-time instead of on-chain).

Do all three:

1. Keep the methods exported (no removal — removal waits for 3.0.0 per the deprecation policy).

2. Add `@deprecated` JSDoc in the `.d.ts`:

```
``ts
```

```
/**
```

```
 * @deprecated Reverts on all deployed markets. Use REST getUserOrders instead.
```

```
 * Will be removed in 3.0.0 per the deprecation policy.
```

```
 */
```

```
``
```

3. Throw synchronously **before** any network call:

```
``ts
```

```
throw new RainNotSupportedError(
```

```
'getUserActiveBuyOrders reverts on deployed markets; use getUserOrders (REST)'
```

```
);
```

```
``
```

Alternative (also non-breaking): if the underlying ABI/contract mismatch can be fixed so the reads actually succeed, that is strictly better — a method going from "always reverts" to "works" breaks nothing.

Effort: ~2 hours (guard path) · **Risk:** none

2. F-02 — Align declared types with the wire **(use additive normalization, not renames)**

!! Do NOT simply rename the declared fields — any consumer already coded against the (broken) declared names would get a compile break. Instead, **normalize in the wrapper and declare both fields**:

| Method | Declared today | Wire returns | Non-breaking fix |

|---|---|---|---|

| `getAllPoolsCount` | `openPoolsCount` | `poolsCount` | Wrapper sets **both**: `data.openPoolsCount = data.poolsCount`. Type declares `poolsCount: number` and **deprecated alias** of `poolsCount`

openPoolsCount: number`. |

| `getTotalPredictionsByUser` | `totalPredictions` | `totalPoolsInvested` | Same pattern: populate both, deprecate the old name, remove alias in 3.0.0. |

Result: code written against the wire works, code written against the old (broken) types ****starts working** instead of getting `undefined` ****** — a pure improvement, zero breaks.

****Prerequisite:**** one verification run of each wrapper against the dev API to confirm the wire shape before shipping.

****Effort:**** ~half day · ****Risk:**** none (additive fields only)

[RECOMMENDED] Strongly recommended — makes the 9 durable

3. REST contract test in CI *(tooling only — zero runtime impact)*

One CI job that calls every REST wrapper against the development API and asserts the response matches the declared type. This:

- resolves F-02 with evidence, permanently
- prevents every future type/wire mismatch from shipping
- is the first thing a re-review will ask about

****Effort:**** ~1 day · ****Risk:**** none (does not ship in the package)

4. F-05 residue — type the remaining event fields *(additive with index signatures)*

Still `Record<string, any>` in `RainSocket.d.ts` (6 occurrences): `enterOption`, `exitOption`, `split`, `enterLiquidity`, etc. Plus `RainAA.client: any`.

Use the exact pattern already shipped for `Pool`/`SubPool`: minimal interface + index signature (`[extra: string]: unknown`). An index-signature interface is a ****superset of `Record<string, any>` access patterns**** — existing dynamic field access keeps compiling.

```ts

```
interface EnterOptionData {
 user?: `0x${string}`;
 option?: number;
 amount?: string;
 [extra: string]: unknown; // <- keeps every existing access working
}
```

```

****Effort:**** ~1 day · ****Risk:**** minimal (verify no `noImplicitAny` friction; index signature preserves compatibility)

[OPTIONAL] Optional — pushes toward 9.5, not required for 9

| Item | Change | Non-breaking approach |

---|---|---

| F-13 | `rain.refreshToken()` | New method, purely additive. Requires backend endpoint first. |

| F-12 | Drop ethers, port allowance helpers to viem | Internal-only if no ethers types leak into public signatures — verify with api-extractor first. If any leak, defer to 3.0.0. |

| F-10 lock | api-extractor pass on release | Tooling only, zero runtime impact. |

Release plan

...

2.3.1 (patch, days):

[x] F-01 deprecate + synchronous RainNotSupportedError guard

[x] F-02 additive normalization (both field names populated, old ones @deprecated)

[x] F-05 residue: 6 event payload interfaces + RainAA.client

[x] CI: REST contract test + api-extractor pass

[-] No removals. No renames. No changed behavior for any working code path.

...

Compatibility statement for the changelog: **"2.3.1 contains no breaking changes. Deprecated aliases remain populated until 3.0.0. The only behavioral change affects two methods that could never succeed (F-01), which now fail synchronously with a descriptive error instead of reverting on-chain."**

Verification

When 2.3.1 is on npm, request the re-verification pass. Expected outcome:

| Dimension | 2.3.0 | Projected 2.3.1 |

---|---|---

| Type safety | 7.5 | 8.5 |

| Runtime correctness | 7.5 | 9 |

| Trust & maturity | 7.5 | 8 |

| **Overall** | **8.5** | **9** |

At 2.3.1 every critical and high finding from the original review is closed — the definition of the 9 band.